운영체제

Assignment#2

담당교수님: 김태석 교수님

학 번 : 2024402041

이 름 : 심 수 정

1. Introduction

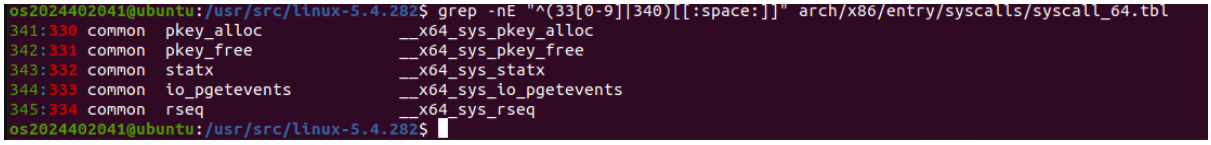
본 과제의 목표는 커널 메시지 큐 시스템 콜 os_mq를 구현하고, 사용자 프로그램의 호출 결과를 후킹 로그와 통계 리포트로 확인하는 것이다. 이를 위해 시스템 콜 등록, 커널 빌드 및 설치, 대화형 셸 실행, 후킹 모듈 적재와 제거를 단계적으로 수행하였다.

메시지 큐는 서로 다른 프로세스가 데이터를 주고받기 위한 일종의 우체통이라고 할 수 있다. 송신 프로세스가 메시지 큐에 데이터를 저장하면 수신 프로세스가 이를 꺼내는 방식으로 동작하며, 먼저 저장된 메시지가 먼저 전달되는 FIFO(First In First Out) 구조를 따른다. 또한 메시지 큐는 커널에서 관리하므로 프로세스 간 직접적인 연결 없이도 데이터를 주고받을 수 있다.

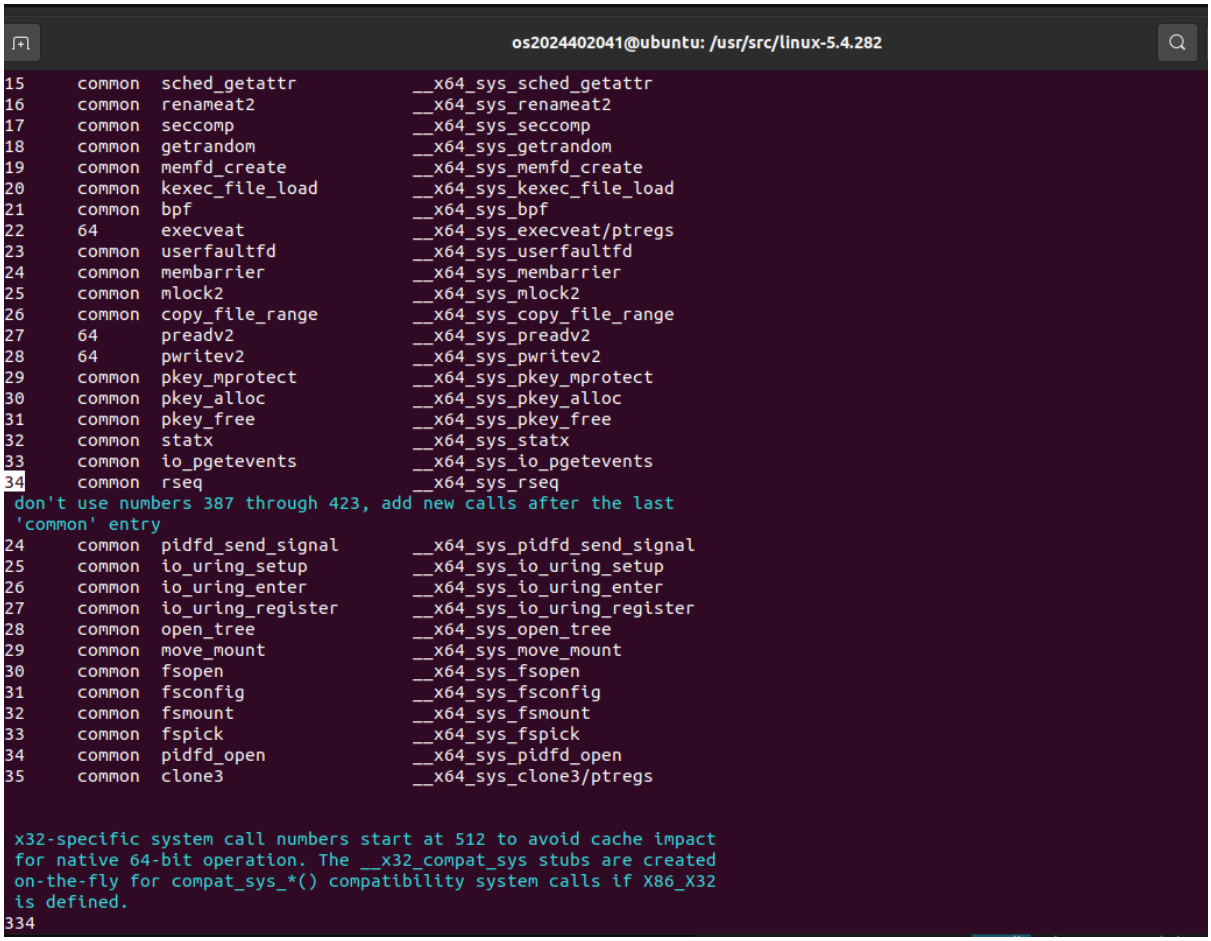
Assignment 2-1에서는 336번 시스템 콜에 OPEN, SEND, RECV, CLOSE 기능을 연결하고 mqsh로 정상 송수신과 FIFO 순서를 확인하였다. 두 터미널에서 빈 큐의 수신 대기와 가득 찬 큐의 송신 대기를 재현하고, 상대 프로세스의 동작으로 대기가 해제되는지 확인하였다. 또한 NONBLOCK 명령이 대기 없이 오류를 반환하는지 비교하였다. Assignment 2-2에서는 호출 명령과 PID를 커널 로그로 관찰하면서 원래의 송수신 기능이 유지되는지 확인하였다. Assignment 2-3에서는 수집 모듈과 리포트 모듈을 분리하고, 실행한 명령 횟수와 요약 통계가 일치하는지 검증하였다.

2. Result

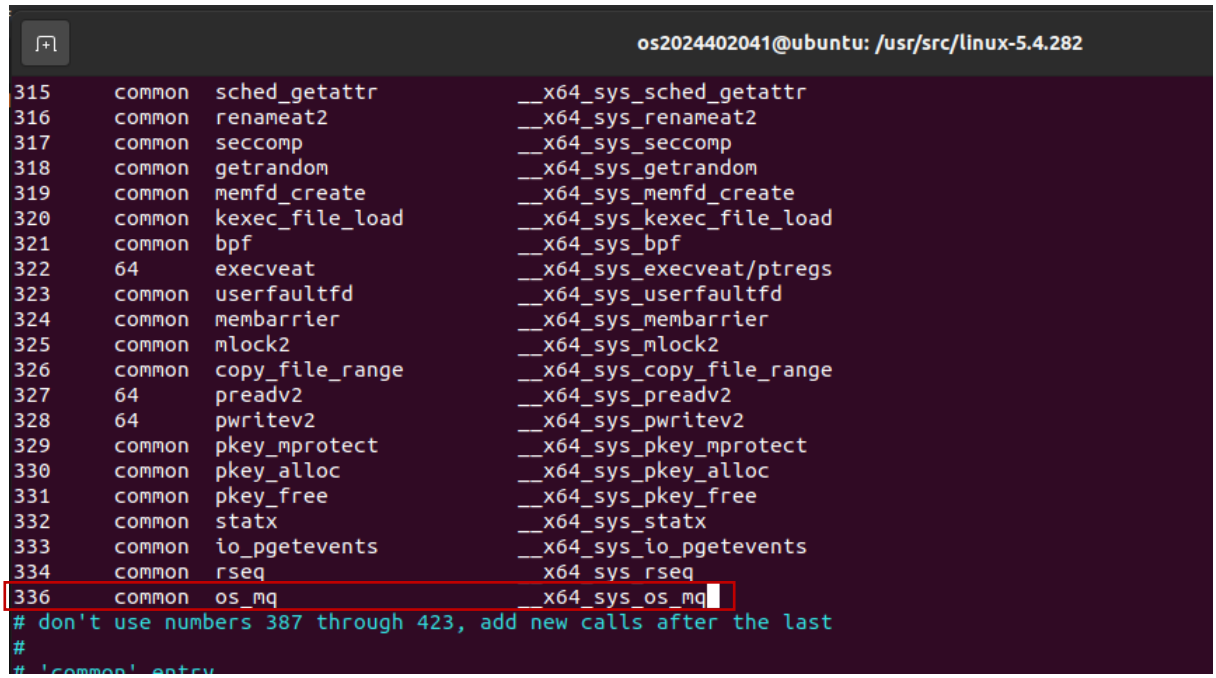
A. Assignment 2-1: 커널 메시지 큐 시스템 콜



작업 계정은 os2024402041이며 커널 소스 경로는 /usr/src/linux-5.4.282이다. arch/x86/entry/syscalls/syscall_64.tbl에서 grep -nE로 330~340번 항목을 조회하였다. 출력에는 334번 rseq까지 나타나고 336번은 없으므로, 과제에서 지정한 336번에 os_mq를 등록할 수 있음을 확인하였다.



Vi로 arch/x86/entry/syscalls/syscall_64.tbl을 열어 334번 rseq 항목을 찾았다. 시스템 콜 번호와 구현 함수의 연결은 이 파일에서 설정하므로 기존 항목을 유지하면서 새 항목을 추가하였다.



334번 항목 아래에 "336 common os_mq __x64_sys_os_mq"를 추가하였다. 이 항목은 사용자 프로그램의 336번 시스템 콜 호출을 x86-64 진입 함수 __x64_sys_os_mq에 연결한다. mqsh에서도 동일한 번호를 사용해야 구현한 시스템 콜이 실행된다.

```
os2024402041@ubuntu:/usr/src/linux-5.4.282$ cd /usr/src/linux-5.4.282
os2024402041@ubuntu:/usr/src/linux-5.4.282$ grep -n "core-y" Makefile
649:core-y              := usr/
1055:core-y             += kernel/ certs/ mm/ fs/ ipc/ security/ crypto/ block/
1058:                   $(core-y) $(core-m) $(drivers-y) $(drivers-m) \
1069:core-y             := $(patsubst %/, %/built-in.a, $(core-y))
1077:export KBUILD_VMLINUX_OBJS := $(head-y) $(init-y) $(core-y) $(libs-y2) \
os2024402041@ubuntu:/usr/src/linux-5.4.282$
```

최상위 Makefile에서 grep -n "core-y" Makefile을 실행하여 커널에 포함되는 디렉터리 목록을 찾았다. 캡처의 1055번 줄에는 kernel/, certs/, mm/, fs/, ipc/, security/, crypto/, block/이 나열되어 있어 os_mq/를 추가할 위치를 확인할 수 있다.

os2024402041@ubuntu: /usr/src/linux-5.4.282

```
HOST_LIBELF_LIBS = $(shell pkg-config libelf --libs 2>/dev/null || echo -lelf)

ifdef CONFIG_STACK_VALIDATION
  has_libelf := $(call try-run,\
                echo "int main() {}" | $(HOSTCC) $(KBUILD_HOSTLDFLAGS) -xc -o /dev/null $(HOST_LIBELF_LIBS) -,1,0)
  ifeq ($(has_libelf),1)
    objtool_target := tools/objtool FORCE
  else
    SKIP_STACK_VALIDATION := 1
    export SKIP_STACK_VALIDATION
  endif
endif

PHONY += prepare0

export MODORDER := $(extmod-prefix)modules.order

ifeq ($(KBUILD_EXTMOD),)
core-y          += kernel/ certs/ mm/ fs/ ipc/ security/ crypto/ block/

vmlinux-dirs    := $(patsubst %/,%,$(filter %/, $(init-y) $(init-m) \
                     $(core-y) $(core-m) $(drivers-y) $(drivers-m) \
                     $(net-y) $(net-m) $(libs-y) $(libs-m) $(virt-y)))

vmlinux-alldirs := $(sort $(vmlinux-dirs) Documentation \
                     $(patsubst %/,%,$(filter %/, $(init-) $(core-) \
                        $(drivers-) $(net-) $(libs-) $(virt-))))

build-dirs      := $(vmlinux-dirs)
clean-dirs      := $(vmlinux-alldirs)

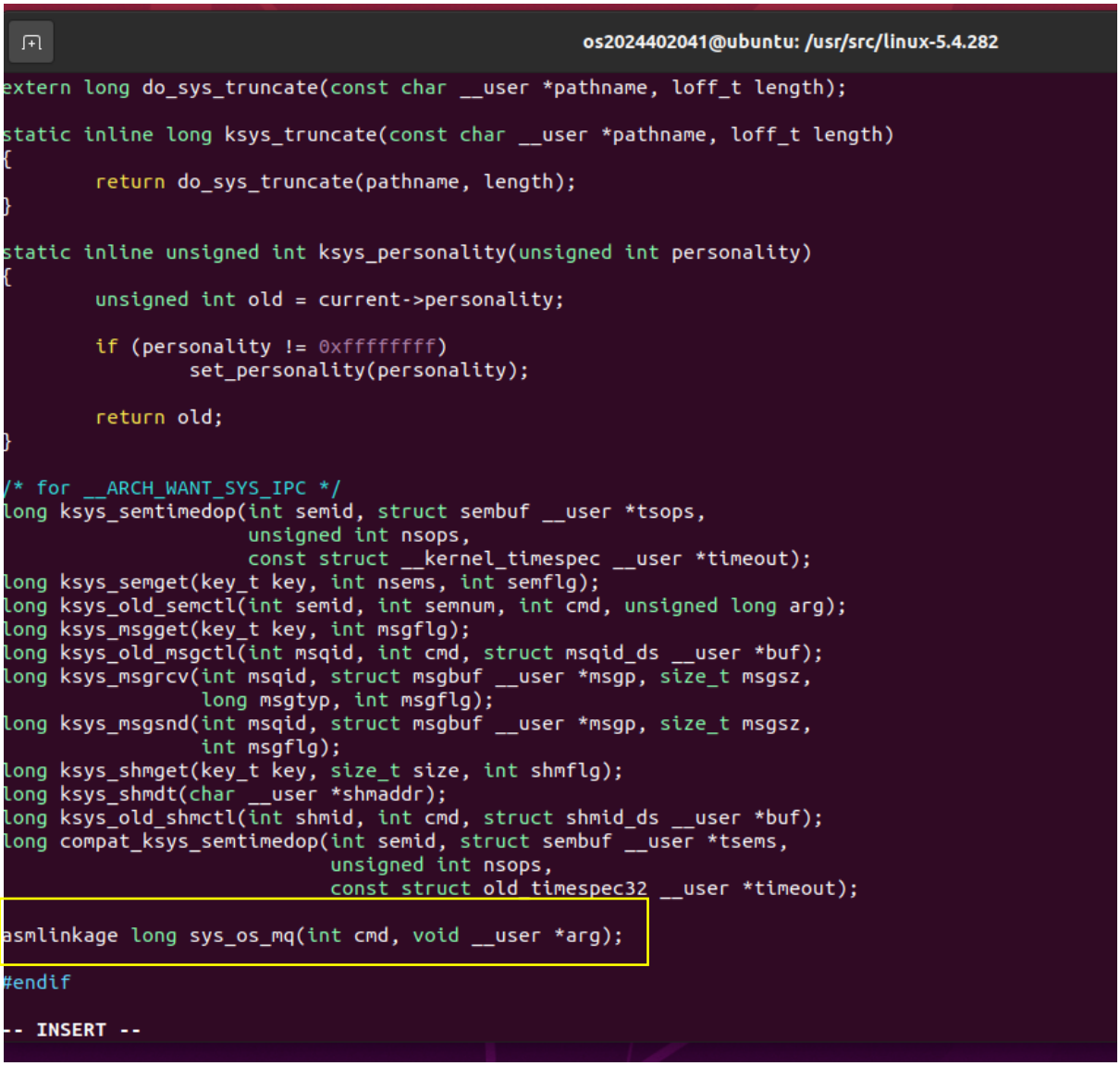
init-y          := $(patsubst %/, %/built-in.a, $(init-y))
core-y          := $(patsubst %/, %/built-in.a, $(core-y))
drivers-y       := $(patsubst %/, %/built-in.a, $(drivers-y))
net-y           := $(patsubst %/, %/built-in.a, $(net-y))
libs-y1         := $(patsubst %/, %/lib.a, $(libs-y))
libs-y2         := $(patsubst %/, %/built-in.a, $(filter-out %.a, $(libs-y)))
virt-y          := $(patsubst %/, %/built-in.a, $(virt-y))
```

1055,1

```
ifeq ($(KBUILD_EXTMOD),)
core-y          += kernel/ certs/ mm/ fs/ ipc/ security/ crypto/ block/ os_mq/

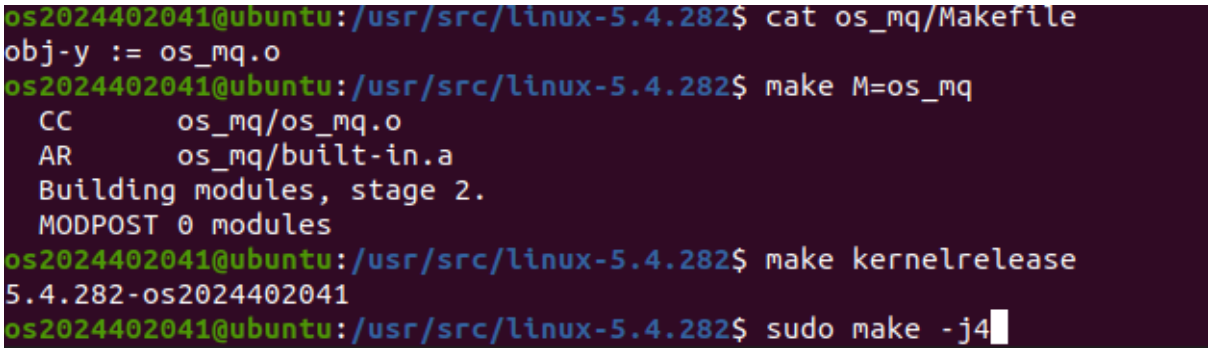
vmlinux-dirs    := $(patsubst %/,%,$(filter %/, $(init-y) $(init-m) \
                     $(core-y) $(core-m) $(drivers-y) $(drivers-m) \
                     $(net-y) $(net-m) $(libs-y) $(libs-m) $(virt-y)))
```

최상위 Makefile의 1055번 줄 core-y 목록 끝에 os_mq/를 추가하였다. 이 설정으로 전체 커널 빌드에 메시지 큐 구현 디렉터리가 포함된다.



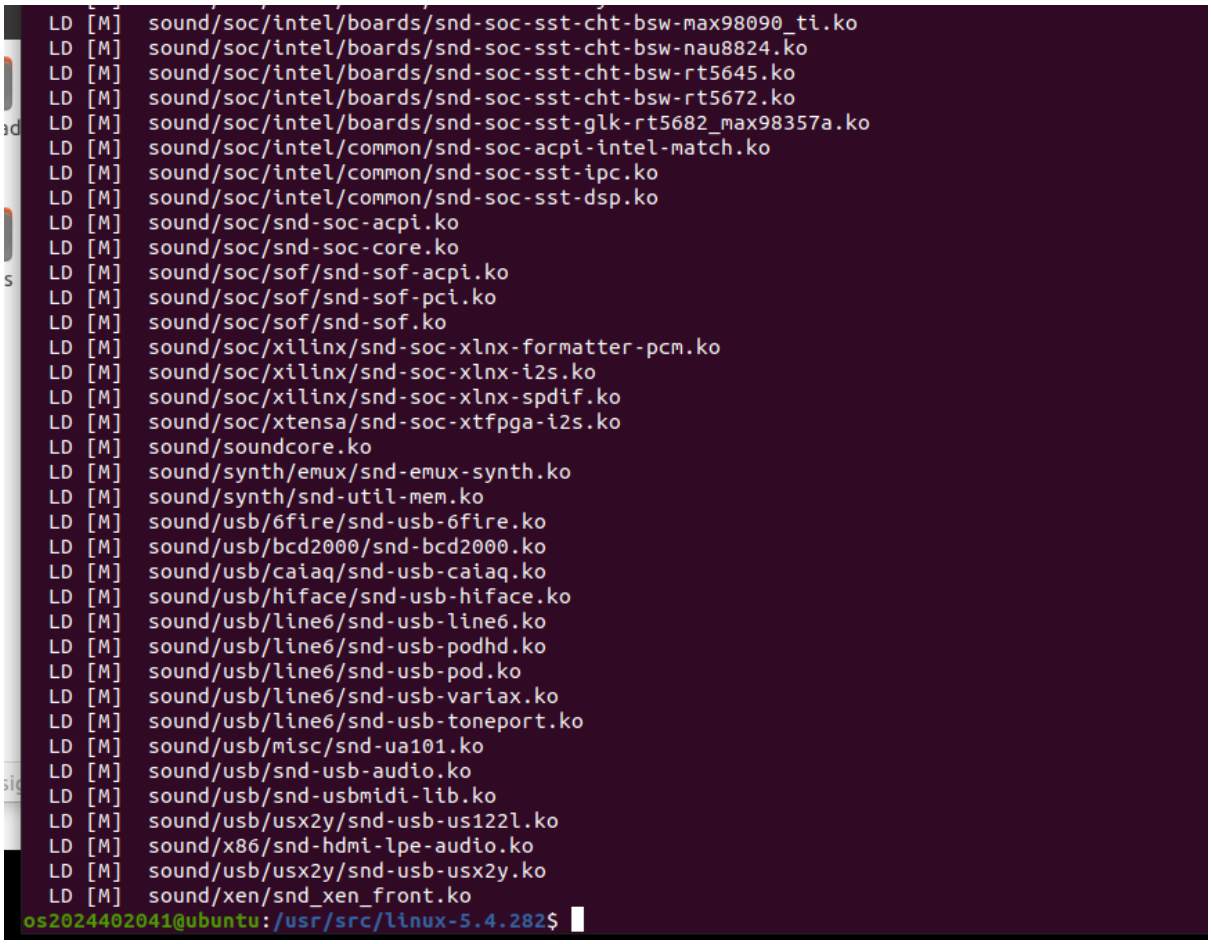
include/linux/syscalls.h의 마지막 #endif 앞에 "asmlinkage long sys_os_mq(int cmd, void __user *arg);"를 선언하였다. cmd는 OPEN, SEND, RECV, CLOSE를 구분하고 arg는 각 명령의 인자 구조체를 전달한다.

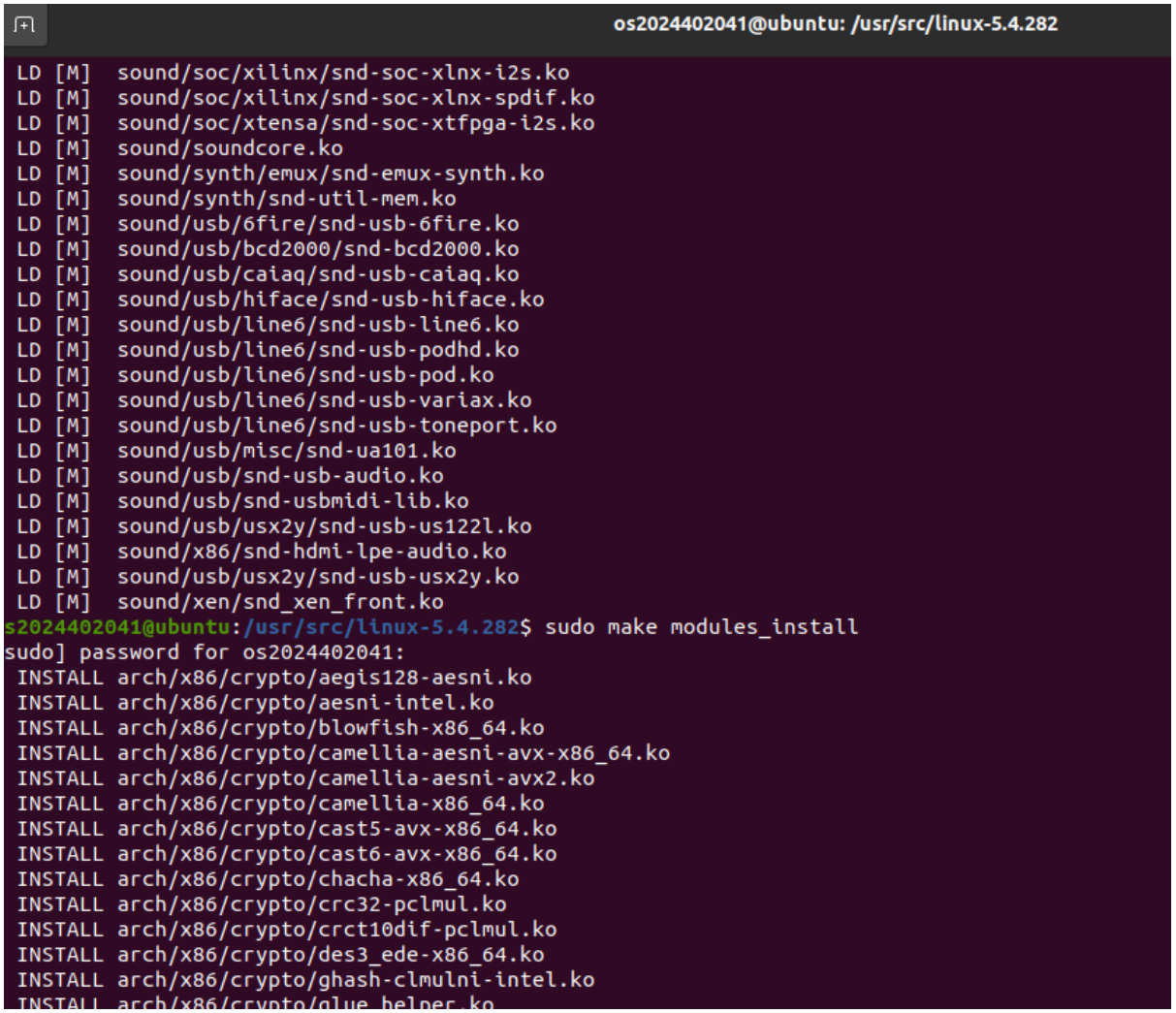
.

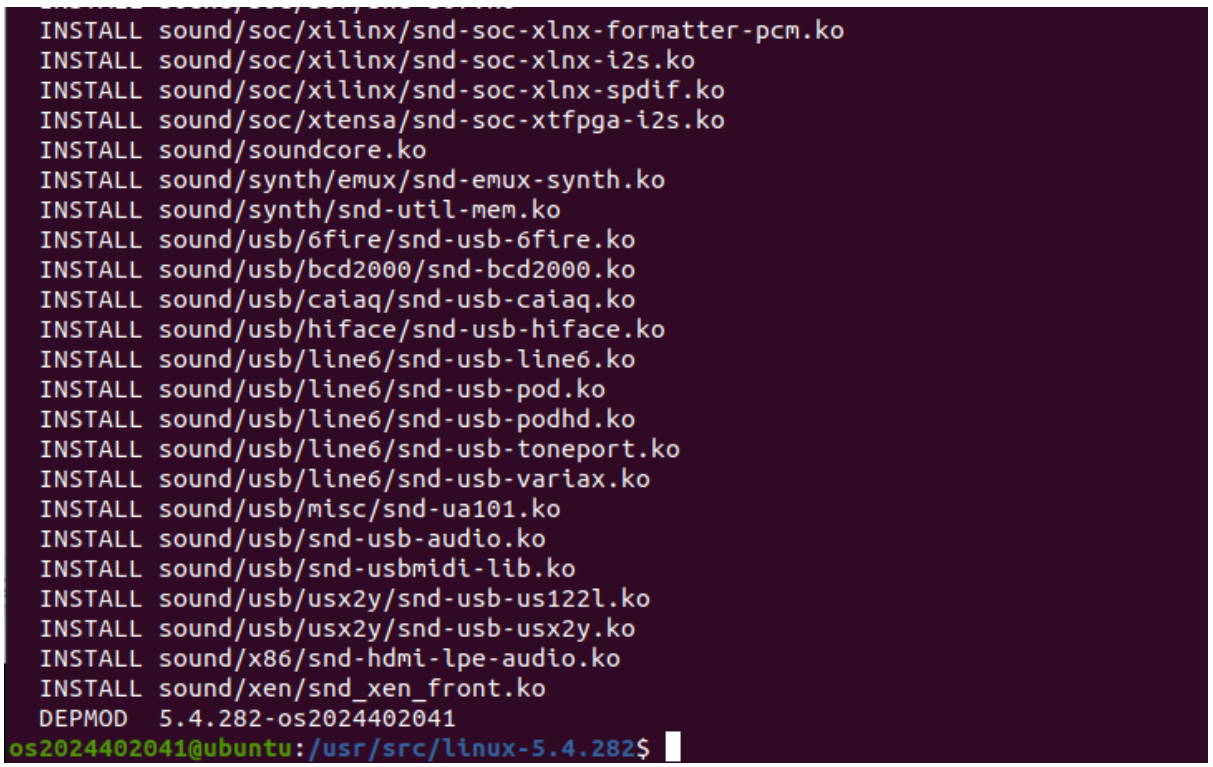


os_mq/Makefile의 "obj-y := os_mq.o"를 확인한 뒤 make M=os_mq로 해당 디렉터리를

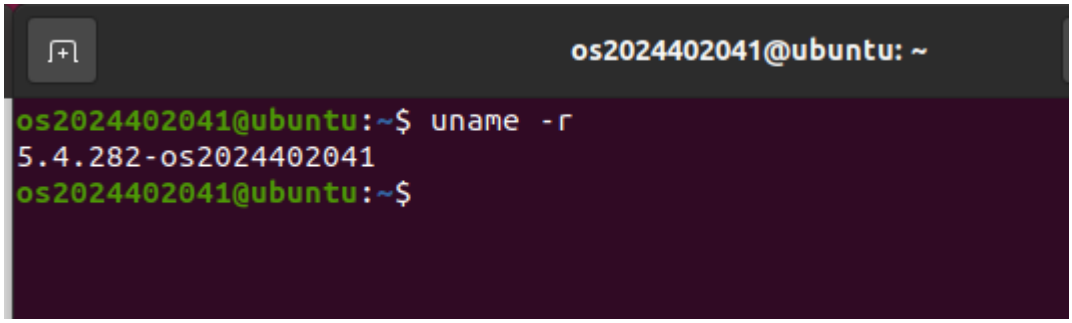
빌드하였다. CC와 AR 출력으로 os_mq.o 및 built-in.a 생성 과정을 확인하였고, make kernelrelease 결과는 5.4.282-os2024402041이었다. 이어 sudo make -j4로 전체 커널을 빌드하였다.




os2024402041@ubuntu: /usr/src/linux-5.4.282
LD [M] sound/soc/xilinx/snd-soc-xlnx-i2s.ko
LD [M] sound/soc/xilinx/snd-soc-xlnx-spdif.ko
LD [M] sound/soc/xtensa/snd-soc-xtfpga-i2s.ko
LD [M] sound/soundcore.ko
LD [M] sound/synth/emux/snd-emux-synth.ko
LD [M] sound/synth/snd-util-mem.ko
LD [M] sound/usb/6fire/snd-usb-6fire.ko
LD [M] sound/usb/bcd2000/snd-bcd2000.ko
LD [M] sound/usb/caiaq/snd-usb-caiaq.ko
LD [M] sound/usb/hiface/snd-usb-hiface.ko
LD [M] sound/usb/line6/snd-usb-line6.ko
LD [M] sound/usb/line6/snd-usb-podhd.ko
LD [M] sound/usb/line6/snd-usb-pod.ko
LD [M] sound/usb/line6/snd-usb-variax.ko
LD [M] sound/usb/line6/snd-usb-toneport.ko
LD [M] sound/usb/misc/snd-ua101.ko
LD [M] sound/usb/snd-usb-audio.ko
LD [M] sound/usb/snd-usbmidi-lib.ko
LD [M] sound/usb/usx2y/snd-usb-us122l.ko
LD [M] sound/x86/snd-hdmi-lpe-audio.ko
LD [M] sound/usb/usx2y/snd-usb-usx2y.ko
LD [M] sound/xen/snd_xen_front.ko
s2024402041@ubuntu:/usr/src/linux-5.4.282$ sudo make modules_install
sudo] password for os2024402041:
INSTALL arch/x86/crypto/aegis128-aesni.ko
INSTALL arch/x86/crypto/aesni-intel.ko
INSTALL arch/x86/crypto/blowfish-x86_64.ko
INSTALL arch/x86/crypto/camellia-aesni-avx-x86_64.ko
INSTALL arch/x86/crypto/camellia-aesni-avx2.ko
INSTALL arch/x86/crypto/camellia-x86_64.ko
INSTALL arch/x86/crypto/cast5-avx-x86_64.ko
INSTALL arch/x86/crypto/cast6-avx-x86_64.ko
INSTALL arch/x86/crypto/chacha-x86_64.ko
INSTALL arch/x86/crypto/crc32-pclmul.ko
INSTALL arch/x86/crypto/crct10dif-pclmul.ko
INSTALL arch/x86/crypto/des3_ede-x86_64.ko
INSTALL arch/x86/crypto/ghash-clmulni-intel.ko


INSTALL sound/soc/xilinx/snd-soc-xlnx-formatter-pcm.ko
INSTALL sound/soc/xilinx/snd-soc-xlnx-i2s.ko
INSTALL sound/soc/xilinx/snd-soc-xlnx-spdif.ko
INSTALL sound/soc/xtensa/snd-soc-xtfpga-i2s.ko
INSTALL sound/soundcore.ko
INSTALL sound/synth/emux/snd-emux-synth.ko
INSTALL sound/synth/snd-util-mem.ko
INSTALL sound/usb/6fire/snd-usb-6fire.ko
INSTALL sound/usb/bcd2000/snd-bcd2000.ko
INSTALL sound/usb/caiaq/snd-usb-caiaq.ko
INSTALL sound/usb/hiface/snd-usb-hiface.ko
INSTALL sound/usb/line6/snd-usb-line6.ko
INSTALL sound/usb/line6/snd-usb-pod.ko
INSTALL sound/usb/line6/snd-usb-podhd.ko
INSTALL sound/usb/line6/snd-usb-toneport.ko
INSTALL sound/usb/line6/snd-usb-variax.ko
INSTALL sound/usb/misc/snd-ua101.ko
INSTALL sound/usb/snd-usb-audio.ko
INSTALL sound/usb/snd-usbmidi-lib.ko
INSTALL sound/usb/usx2y/snd-usb-us122l.ko
INSTALL sound/usb/usx2y/snd-usb-usx2y.ko
INSTALL sound/x86/snd-hdmi-lpe-audio.ko
INSTALL sound/xen/snd_xen_front.ko
DEPMOD 5.4.282-os2024402041
os2024402041@ubuntu:/usr/src/linux-5.4.282$

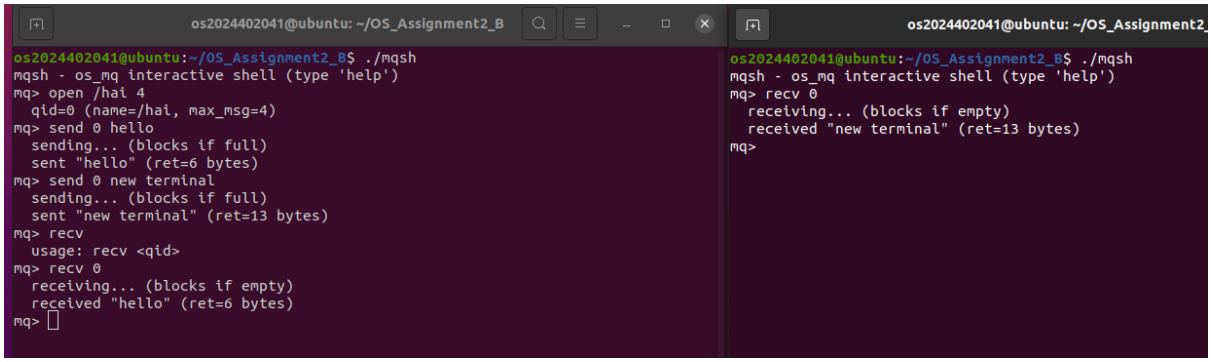
```
s2024402041@ubuntu:~/OS_Assignment2_B$ cd /usr/src/linux-5.4.282
s2024402041@ubuntu:/usr/src/linux-5.4.282$ sudo make install
sudo] password for os2024402041:
h ./arch/x86/boot/install.sh 5.4.282-os2024402041 arch/x86/boot/bzImage \
       System.map "/boot"
un-parts: executing /etc/kernel/postinst.d/initramfs-tools 5.4.282-os2024402041
/boot/vmlinuz-5.4.282-os2024402041
pdate-initramfs: Generating /boot/initrd.img-5.4.282-os2024402041
```



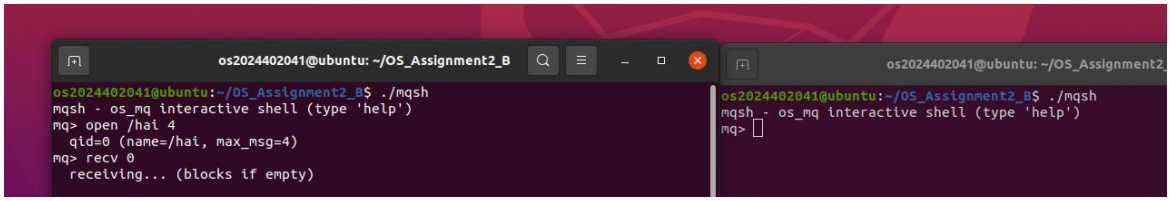
빌드 완료 후 sudo make modules_install로 모듈을 설치하였다. sudo make install과 재부팅 후 uname -r 결과가 5.4.282-os2024402041로 나타나 수정한 커널로 부팅했음을 확인하였다.

```
os2024402041@ubuntu:~/OS_Assignment2_B$ ./mqsh
mqsh - os_mq interactive shell (type 'help')
mq> open /hai 4
qid=0 (name=/hai, max_msg=4)
mq> send 0 hello
sent "hello" (ret=5 bytes)
mq> send 0 new terminal
sent "new terminal" (ret=12 bytes)
mq> recv 0
received "hello" (ret=5 bytes)
mq>
```

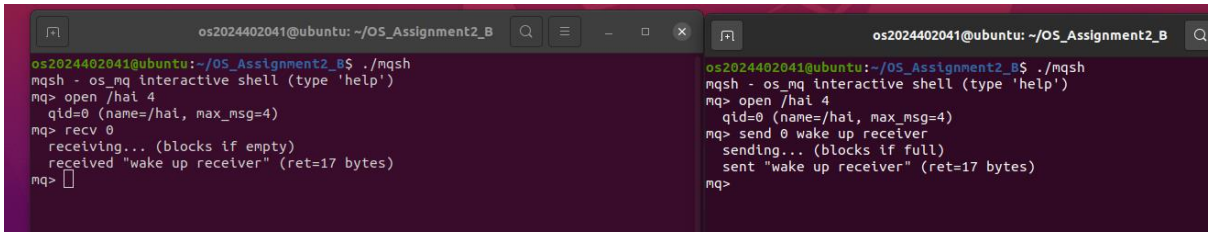
크기가 4인 큐를 생성해 hello와 new terminal을 보내고 꺼내니 가장 처음 보낸 hello가 나왔다. 정상적인 큐의 FIFO 동작이 이루어짐을 확인하였다.



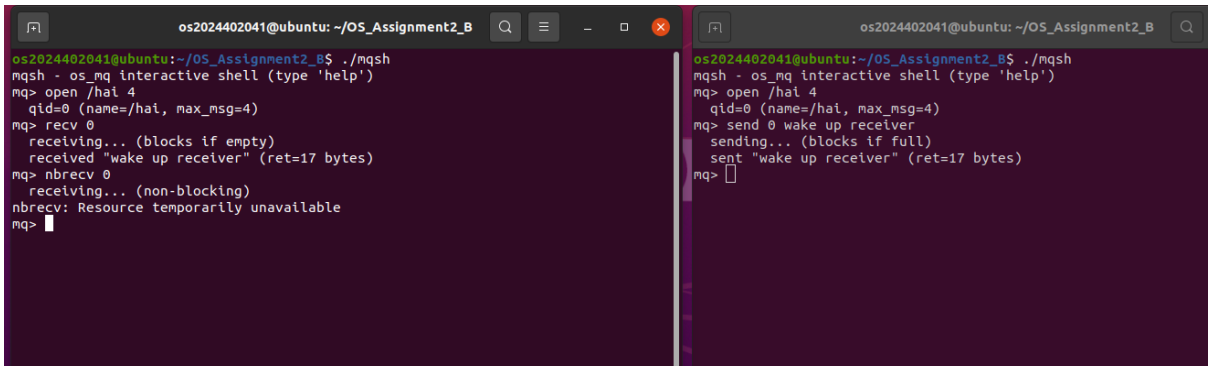
open /hai 4를 실행하고 send 0 hello와 send 0 new terminal로 메시지를 순서대로 저장한 뒤 첫 번째 터미널의 recv 0에서는 "hello", 두 번째 터미널의 recv 0에서는 "new terminal"이 수신되었다. 따라서 서로 다른 프로세스가 같은 커널 큐를 이용하며 먼저 저장된 메시지가 먼저 수신되는 FIFO 동작을 확인하였다. qid 없이 recv만 입력했을 때는 "usage: recv <qid>"가 출력되었다.



빈 큐에서 첫 번째 터미널이 recv 0을 실행하자 "receiving... (blocks if empty)"가 출력된 후 mq> 프롬프트가 돌아오지 않았다. 수신할 메시지가 없어 호출이 완료되지 않고 대기하는 상태를 보여 준다.

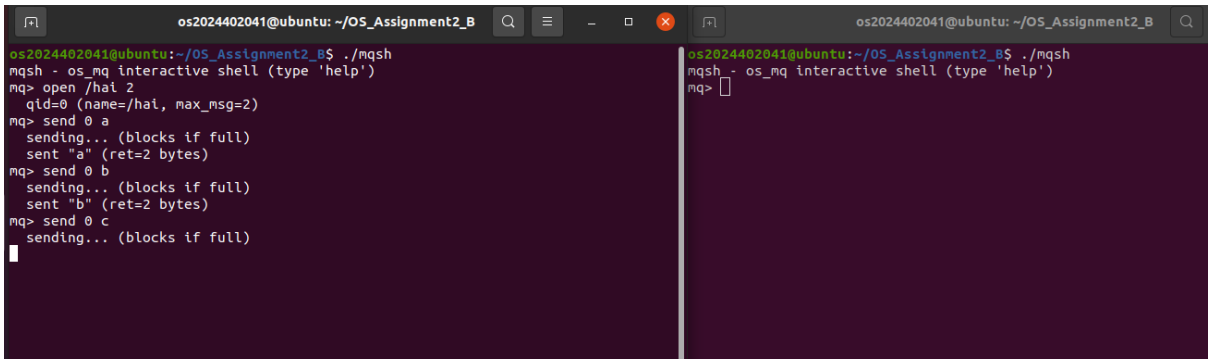


두 번째 터미널에서도 open /hai 4를 실행하자 동일한 qid=0이 반환되었다. send 0 wake up receiver를 실행하니 대기하던 첫 번째 터미널이 "wake up receiver"를 수신하고 프롬프트로 복귀하였다. 같은 이름의 큐를 공유하며 송신으로 메시지가 추가되면 수신 대기가 해제됨을 확인하였다.

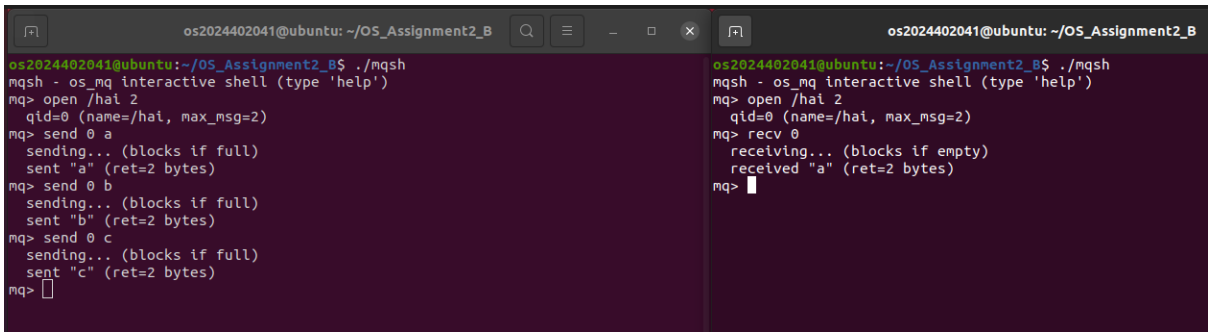


메시지를 모두 수신한 후 nbrecv 0을 실행하자 "Resource temporarily unavailable"이 출

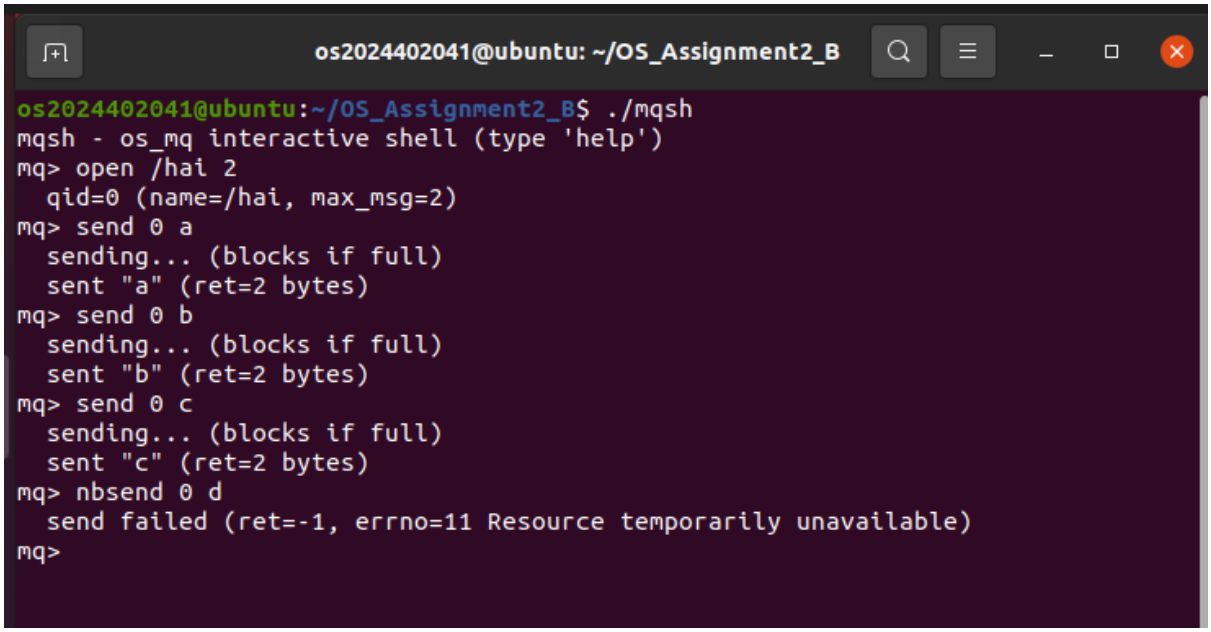
력되고 프롬프트가 즉시 돌아왔다. 일반 recv와 달리 메시지가 들어올 때까지 기다리지 않았으며, 빈 큐에 대한 NONBLOCK 호출이 EAGAIN 오류로 종료되는 동작과 일치한다.



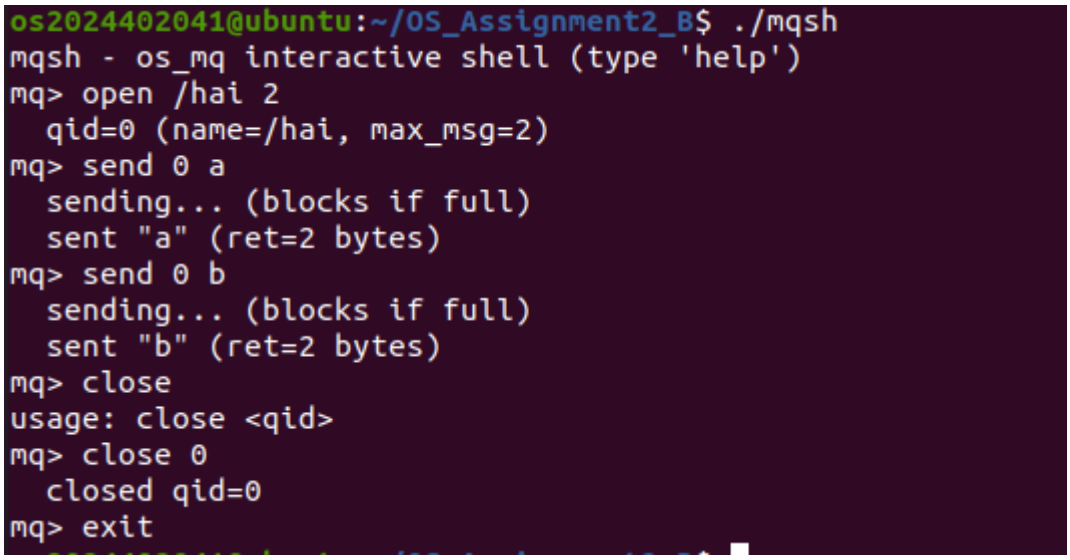
open /hai 2로 최대 메시지 수를 2개로 설정하였다. send 0 a와 send 0 b는 각각 성공했지만 세 번째 send 0 c에서는 성공 메시지와 프롬프트가 나타나지 않았다. 큐가 가득 차 수신으로 빈자리가 생기기 전까지 송신 호출이 대기하는 상태이다.



두 번째 터미널에서 같은 이름으로 open /hai 2를 실행하고 recv 0으로 "a"를 꺼내자 첫 번째 터미널의 send 0 c가 완료되었다. "sent"에 해당하는 성공 출력과 프롬프트가 나타났다. 수신으로 한 자리가 확보되면 대기하던 송신이 재개됨을 확인하였다. 이때 큐에는 "b"와 "c"가 남으므로 다시 가득 찬 상태이다.



이 상태에서 nbsend 0 d를 실행하자 "send failed (ret=-1, errno=11 Resource temporarily unavailable)"이 즉시 출력되었다. 커널의 -EAGAIN은 사용자 공간의 syscall 호출에서 반환값 -1과 errno=11로 전달된다. 따라서 NONBLOCK 송신은 빈자리가 생기기를 기다리지 않으며 가득 찬 큐의 Blocking 송신과 구별된다.

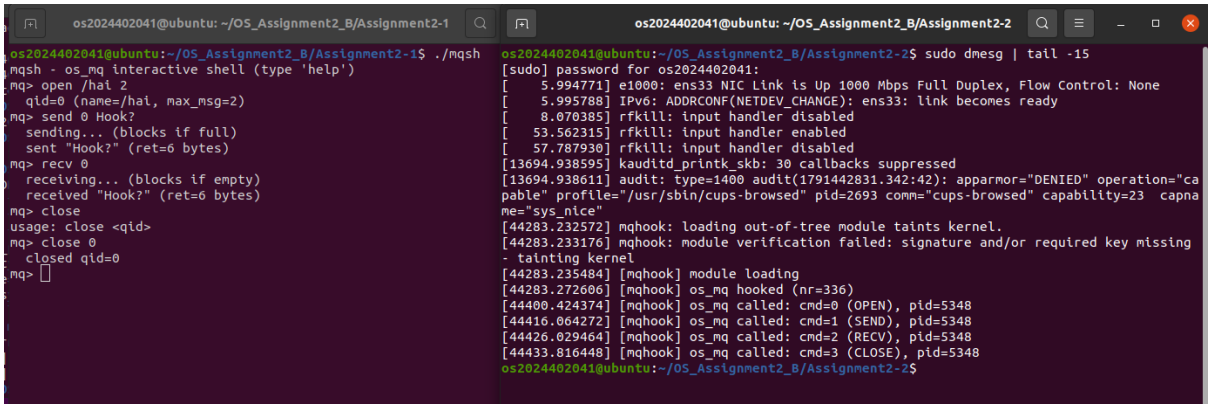


메시지 두 개를 저장한 뒤 close만 입력하자 "usage: close <qid>"가 출력되었다. close 0을 입력했을 때는 "closed qid=0"이 출력되었다. 따라서 캡처의 셸은 qid를 명시하는 종료 형식을 사용한다. 제안서 8쪽에는 전체 큐 정리로 close가 소개되지만 11쪽 화면 캡쳐에서는 qid를 입력받아 초기화하게 되어있으므로 11쪽에 따라 qid를 요구하도록 하였다.

B. Assignment 2-2: os_mq 시스템 콜 후킹(호출 로그 관찰)

```
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-2$ make
make -C /lib/modules/5.4.282-os2024402041/build M=/home/os2024402041/OS_Assignme
nt2_B/Assignment2-2 modules
make[1]: Entering directory '/usr/src/linux-5.4.282'
  CC [M]  /home/os2024402041/OS_Assignment2_B/Assignment2-2/mqhook.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC [M]  /home/os2024402041/OS_Assignment2_B/Assignment2-2/mqhook.mod.o
  LD [M]  /home/os2024402041/OS_Assignment2_B/Assignment2-2/mqhook.ko
make[1]: Leaving directory '/usr/src/linux-5.4.282'
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-2$ sudo insmod mqhook.ko
[sudo] password for os2024402041:
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-2$
```

Assignment2-2 디렉터리에서 make를 실행하여 mqhook.ko를 생성하였다. /lib/modules/5.4.282-os2024402041/build 경로는 실행 중인 커널의 빌드 환경을 사용했음을 보여 준다. 이어 sudo insmod mqhook.ko로 후킹 모듈을 적재하였다.



mqsh에서 open /hai 2, send 0 Hook?, recv 0, close 0을 실행하고 sudo dmesg | tail -15로 로그를 확인하였다. "os_mq hooked (nr=336)" 다음에 cmd=0 (OPEN), cmd=1 (SEND), cmd=2 (RECV), cmd=3 (CLOSE)가 순서대로 출력되었다. "Hook?"의 송신과 수신도 각각 6바이트로 성공하여 후킹 중에도 원래 큐 기능이 유지됨을 확인하였다. close만 입력한 명령은 사용법 출력으로 처리되며 로그의 CLOSE는 close 0에 대응한다. 모듈 서명 관련 경고 뒤에도 후킹과 호출 로그가 출력되어 이 실행에서는 모듈이 적재되었음을 알 수 있다.

```
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-2$ sudo rmmod mqhook
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-2$ sudo dmesg | tail -10
[44283.232572] mqhook: loading out-of-tree module taints kernel.
[44283.233176] mqhook: module verification failed: signature and/or required key missing
- tainting kernel
[44283.235484] [mqhook] module loading
[44283.272606] [mqhook] os_mq hooked (nr=336)
[44400.424374] [mqhook] os_mq called: cmd=0 (OPEN), pid=5348
[44416.064272] [mqhook] os_mq called: cmd=1 (SEND), pid=5348
[44426.029464] [mqhook] os_mq called: cmd=2 (RECV), pid=5348
[44433.816448] [mqhook] os_mq called: cmd=3 (CLOSE), pid=5348
[44673.128802] [mqhook] os_mq restored
[44673.128803] [mqhook] module unloaded
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-2$
```

sudo rmmod mqhook 실행 후 dmesg에 "os_mq restored"와 "module unloaded"가 출력되었다. 모듈 제거 과정에서 원래 시스템 콜 엔트리로 복원하는 경로가 실행되었음을 보여 준다.

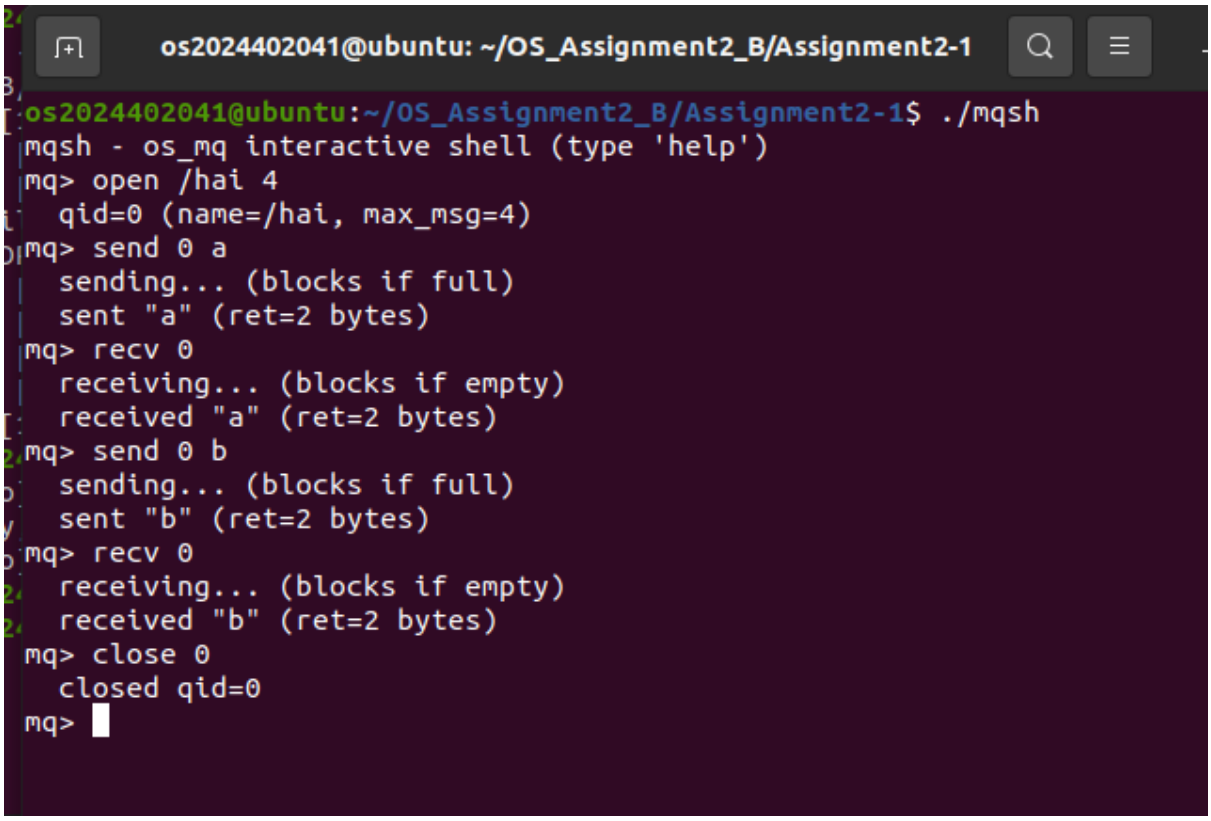
C. Assignment 2-3: 호출 통계 리포트

```
os2024402041@ubuntu: ~/OS_Assignment2_B/Assignment2-3
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-3$ make
make -C /lib/modules/5.4.282-os2024402041/build M=/home/os2024402041/OS_Assignme
nt2_B/Assignment2-3 modules
make[1]: Entering directory '/usr/src/linux-5.4.282'
  CC [M]  /home/os2024402041/OS_Assignment2_B/Assignment2-3/mqhooking.o
  CC [M]  /home/os2024402041/OS_Assignment2_B/Assignment2-3/mqmonitor.o
  Building modules, stage 2.
  MODPOST 2 modules
  CC [M]  /home/os2024402041/OS_Assignment2_B/Assignment2-3/mqhooking.mod.o
  LD [M]  /home/os2024402041/OS_Assignment2_B/Assignment2-3/mqhooking.ko
  CC [M]  /home/os2024402041/OS_Assignment2_B/Assignment2-3/mqmonitor.mod.o
  LD [M]  /home/os2024402041/OS_Assignment2_B/Assignment2-3/mqmonitor.ko
make[1]: Leaving directory '/usr/src/linux-5.4.282'
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-3$
```

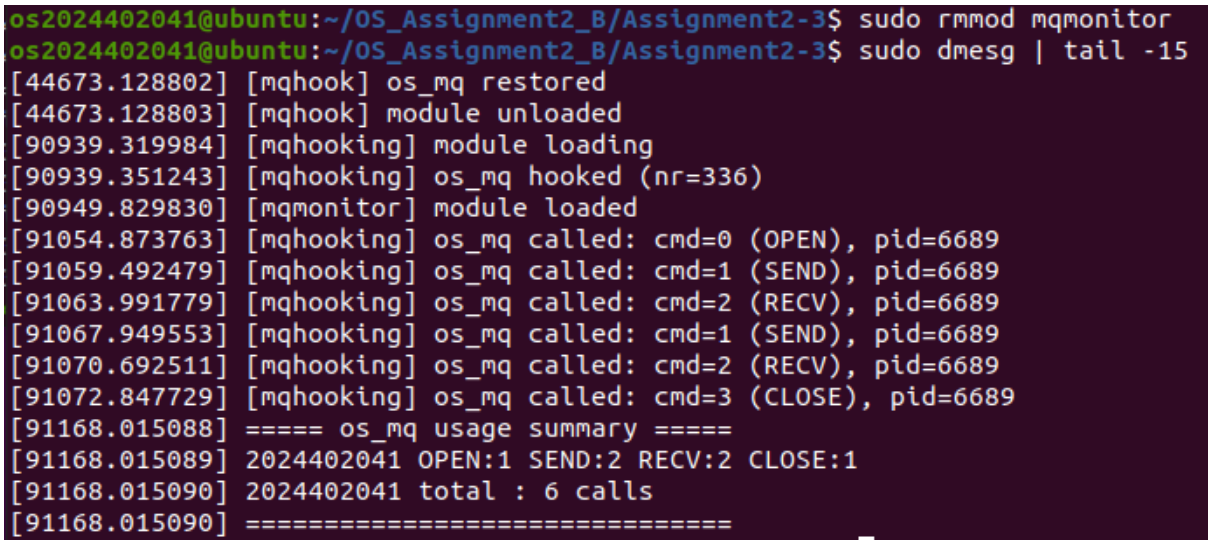
Assignment2-3 디렉터리에서 make를 실행하여 mqhooking.ko와 mqmonitor.ko를 생성하였다.

```
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-3$ sudo insmod mqhooking.ko
[sudo] password for os2024402041:
Sorry, try again.
[sudo] password for os2024402041:
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-3$ sudo insmod mqmonitor.ko
os2024402041@ubuntu:~/OS_Assignment2_B/Assignment2-3$
```

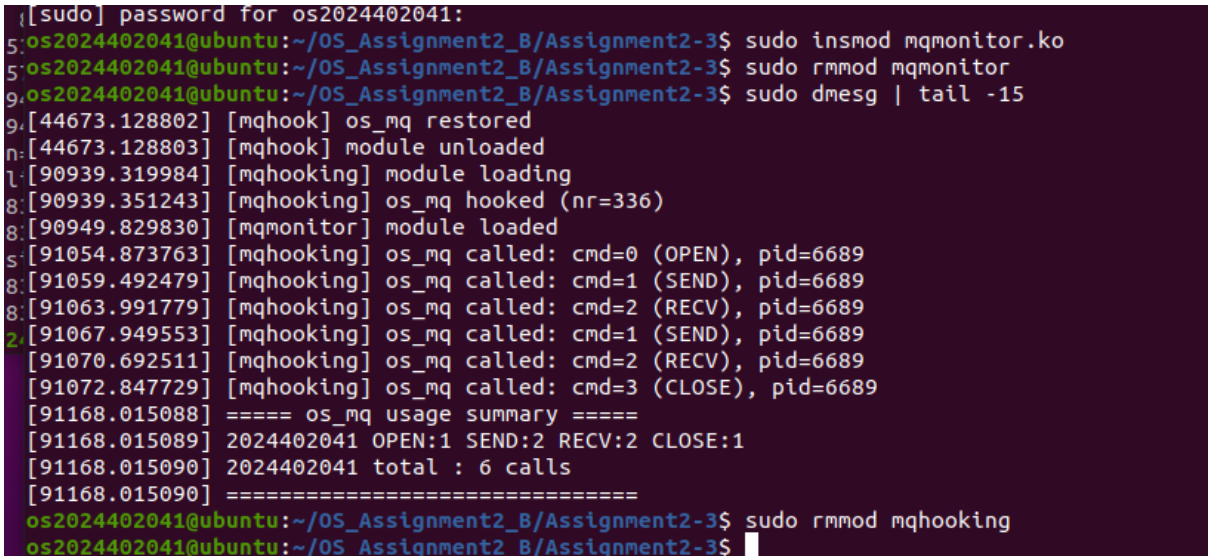
sudo insmod mqhooking.ko를 먼저 실행하고 sudo insmod mqmonitor.ko를 이어 실행하였다. 제안서의 구조에서 mqhooking은 카운터를 수집하고 EXPORT_SYMBOL로 공유하며 mqmonitor는 공유 헤더와 심볼을 통해 통계를 가져온다. 따라서 제공 모듈을 먼저 적재해야 한다.



통계 검증을 위해 open /hai 4를 1회 , send 0 a와 send 0 b를 각각 1회, recv 0을 2회, close 0을 1회 실행하였다. 송수신은 각각 2바이트로 성공하였고 메시지는 "a", "b" 순서로 수신되었다. 예상 호출 횟수는 OPEN 1회, SEND 2회, RECV 2회, CLOSE 1회로 합계 6회이다.



sudo rmmod mqmonitor 후 dmesg에는 PID 6689의 OPEN, SEND, RECV, SEND, RECV, CLOSE 호출이 기록되어 있다. 이어 "os_mq usage summary"와 학번 2024402041, "OPEN:1 SEND:2 RECV:2 CLOSE:1", "total : 6 calls"가 출력되었다. 명령별 횟수와 총합이 예상값과 일치하여 이 실험에서는 호출 수집과 제거 시 요약 출력이 정상 동작하였다.



리포트 모듈 mqmonitor를 먼저 제거한 뒤 sudo rmmod mqhooking으로 수집·후킹 모듈을 제거하였다. 공유 심볼을 참조하는 모듈을 먼저 제거하는 순서는 적재 순서의 역순이다.

3. 고찰

이번 과제를 통해 메시지 큐를 이용한 프로세스 간 통신과 Blocking 동작을 이해할 수 있었다. 특히 큐가 비어 있거나 가득 찬 경우 작업이 대기 상태에 있다가 다른 터미널에서 메시지를 송수신하면 다시 실행되는 것을 직접 확인할 수 있었다. 이를 통해 여러 프로세스가 데이터를 주고받는 과정에서 동기화가 중요하다는 것을 알게 되었다.

또한 시스템 콜을 직접 구현하고 커널을 컴파일하는 과정에서 소스코드뿐만 아니라 시스템 콜 테이블과 Makefile 등의 설정도 중요하다는 것을 알게 되었다. 이후 후킹 모듈을 통해 시스템 콜의 호출 기록을 확인하고, 호출 횟수를 집계하면서 커널 모듈의 동작 방식도 이해할 수 있었다.

후킹된 시스템 콜이 Blocking 상태일 때 모듈을 제거하면, 대기 중인 호출이 제거된 모듈의 코드를 다시 실행하려고 할 수 있다. 따라서 모듈 제거 전에는 대기 중인 호출이 없는지 확인해야 한다.

4. Reference

강의 자료만을 참고.